

# Git Branching, Pull Requests, and Conflicts

```
$ git branch  
$ git switch -c new-branch  
$ git push
```

## Where We Are

By now, we have practiced the basic Git loop:

```
git status  
git add  
git commit  
git push  
git pull
```

Today we add one more important idea:

How do we safely work on changes without disturbing the main project?

# Today's Goals

By the end of this session, you should be able to:

- explain what a branch is
- create and switch branches
- understand why branches are useful
- merge a branch back into `main`
- recognize and resolve a merge conflict
- use a branch-based workflow in your own repo

# Why Branch?

## Analogy: A Safe Copy of Your Workbench

Imagine `main` is the clean, stable version of your project.

A branch is like setting up a separate workbench.

You can experiment, make a mess, and try ideas without touching the stable version.

|                |                            |
|----------------|----------------------------|
| main branch    | = stable workbench         |
| feature branch | = experiment workbench     |
| merge          | = bring finished work back |

# Why Branches Are Useful

Branches help us:

- try ideas safely
- work on one task at a time
- collaborate without overwriting each other
- keep `main` stable
- review changes before merging
- fix problems without disrupting other work

## Common Branch Use Cases

```
feature/add-readme  
fix/typo-in-script  
docs/update-installation  
analysis/test-new-model
```

Each branch should usually have one clear purpose.

# Branches Are Lightweight

A branch is not a full copy of your project.

It is more like a pointer to a commit.

```
main  ---- A ---- B
                ↑
            feature
```

Creating branches is cheap and fast.



# Mental Model

```
Commit = saved checkpoint  
Branch = movable label pointing to a checkpoint  
HEAD   = where you are currently working
```

When you commit on a branch, that branch pointer moves forward.

# Branch Commands

## See Your Branches

```
git branch
```

The branch with `*` is the branch you are currently on.

Example:

```
* main  
  feature-readme
```

## Create a New Branch

```
git switch -c feature-readme
```

This means:

Create a new branch called feature-readme,  
and switch to it.

Older command you may see online:

```
git checkout -b feature-readme
```

## Switch Between Branches

```
git switch main
```

```
git switch feature-readme
```

Older command you may see online:

```
git checkout main
```

## Check the Branch History

```
git log --oneline --graph --decorate --all
```

This gives a compact view of commits and branches.

Useful when you want to see where branches split and merge.

# Best Practices

# Branch Best Practices

Before creating a branch:

```
git switch main  
git pull
```

Then create your new branch:

```
git switch -c docs/update-readme
```

Start from the latest shared version whenever possible.



# Naming Branches

Good branch names are short and descriptive.

```
feature/add-login  
fix/data-loading-error  
docs/update-readme  
analysis/compare-models
```

Avoid vague names:

```
stuff  
changes  
keli-branch  
new
```

# Keep Branches Focused

A good branch usually does one thing.

Better:

```
Branch 1: update README  
Branch 2: clean analysis script  
Branch 3: add visualization
```

Harder to review:

```
One branch with README + script cleanup + new plots + random fixes
```

## Commit Often, But Meaningfully

Good commits are small enough to understand.

```
git add README.md  
git commit -m "Add installation instructions"
```

Avoid one giant commit at the end of a long work session.

# Merging on GitHub

# What Is a Merge?

A merge brings changes from one branch into another branch.

Most often:

```
feature branch → main branch
```

Analogy:

You finished work on the experiment workbench, and now you bring the useful result back to the stable workbench.

# Local Merge vs GitHub Merge

There are two common places where merging can happen:

| Where        | What it means                                   |
|--------------|-------------------------------------------------|
| Local merge  | You run <code>git merge</code> on your computer |
| GitHub merge | You open a Pull Request and merge on GitHub     |

In this course, we will practice **GitHub merge**.

That means:

```
push branch → open Pull Request → review → merge on GitHub
```

# Why Merge on GitHub?

Merging on GitHub is useful because it gives us a shared review space.

Before merging, we can:

- compare the branch against `main`
- review exactly what changed
- discuss changes with collaborators
- catch conflicts before changing `main`
- keep a visible project history

## Visual: Branch Before Pull Request

```
main:      A ---- B
           \
branch:    C ---- D
```

The branch has commits that are not yet in `main`.



## Visual: Pull Request on GitHub

GitHub compares:

branch ————— wants to merge into ————— main

"Can these changes be safely added?"

A Pull Request is a request to bring branch changes into `main`.

## Visual: After GitHub Merge

```
main:      A ---- B ----- M
           \      /
branch:    C ---- D
```

M is the merge result.

After this, `main` includes the branch changes.

# What Is a Pull Request?

A Pull Request, or PR, is a GitHub page for reviewing and merging a branch.

A PR usually shows:

- which branch is merging into which branch
- commits included in the branch
- files changed
- line-by-line differences
- whether the branch can merge cleanly
- whether there are conflicts

# Merge Conflicts

## What Is a Conflict?

A conflict happens when Git cannot automatically decide how to combine changes.

Most common case:

- Two branches changed the same part of the same file in different ways.

GitHub pauses the merge and asks you to choose the final version.

# Analogy: Two People Edit the Same Sentence

Original sentence:

```
This project analyzes data.
```

Branch changes it to:

```
This project analyzes survey data.
```

Main changes it to:

```
This project analyzes GitHub data.
```

GitHub cannot know which one you want.

## What Conflict Markers Look Like

GitHub may show conflict markers like this:

```
<<<<<< main  
This project analyzes GitHub data.  
=====  
This project analyzes survey data.  
>>>>>> update-readme-title
```

Your job is to edit this into the final version you want.

# How to Resolve a Conflict on GitHub

1. Open the Pull Request.
2. Click **Resolve conflicts**.
3. Edit the file into the final version you want.
4. Remove conflict markers:

```
<<<<<<<  
=====  
>>>>>>>
```

5. Click **Mark as resolved**.
6. Commit the resolution.
7. Merge the Pull Request.



# Do Not Panic

A conflict does not mean you broke Git.

It means Git found overlapping changes and needs human judgment.

The key question is:

What should the final file say?

# Code Along

# Code Along Goal

We will work on the same GitHub repo `book` .

Then we will use one file, `chapter1.txt` , to practice:

1. creating a branch
2. pushing the branch to GitHub
3. editing `main` on GitHub
4. opening a Pull Request
5. resolving a conflict on GitHub
6. merging on GitHub
7. pulling the final result back locally

## Code Along Picture

GitHub repo: book

main branch: chapter1.txt has one opening line

feature branch: changes the same opening line

main on GitHub: also changes that same opening line

Result: GitHub asks us to resolve the conflict before merging

## Step 0: Make sure we are in the right starting point

Before We Start, In your terminal:

```
cd book  
git switch main  
git pull  
git status
```

## Step 1: Create `chapter1.txt` on `main`

Create a new file:

```
touch chapter1.txt
```

Open it:

```
code chapter1.txt  
# or: nano chapter1.txt
```

Add this text:

```
Chapter 1: A quiet morning  
  
Once upon a time, a new Git repo was born.
```

Save the file.

## Step 2: Commit and Push `chapter1.txt`

Check what changed:

```
git status  
git diff
```

Stage, commit, and push:

```
git add chapter1.txt  
git commit -m "Add chapter 1"  
git push
```

Remember:

```
git add      = move change to review tray  
git commit  = save it to local history  
git push    = send local commits to GitHub
```

## Step 3: Create One Branch

Create and switch to a new branch:

```
git switch -c revise-chapter1-opening
```

Check where you are:

```
git branch
```

You should see:

```
* revise-chapter1-opening  
main
```



## Step 4: Edit `chapter1.txt` on the Branch

Open `chapter1.txt` again.

Change the first line to:

```
Chapter 1: The Adventure Begins
```

Leave the rest of the file the same.

Save the file.

## Step 5: Commit the Branch Change

Check what changed:

```
git status  
git diff
```

Stage and commit:

```
git add chapter1.txt  
git commit -m "Revise chapter 1 opening on branch"
```

This commit is saved locally on your branch.

GitHub cannot see it yet.

## Step 6: Push the Branch to GitHub

Send the branch to GitHub:

```
git push -u origin revise-chapter1-opening
```

The `-u` connects your local branch to the remote branch on GitHub.

After this, GitHub can see your branch.

## Step 7: Edit `main` on GitHub

Now we intentionally create a conflict.

On GitHub:

1. Go to your `book` repository.
2. Make sure you are viewing the `main` branch.
3. Open `chapter1.txt`.
4. Click the edit pencil.
5. Change the same first line to:

```
Chapter 1: A Different Beginning
```

6. Commit the change directly to `main`.

## Step 8: Open a Pull Request

On GitHub, open a Pull Request:

```
base: main  
compare: revise-chapter1-opening
```

This means:

```
Please merge revise-chapter1-opening into main.
```

GitHub should detect that both branches changed the same line in `chapter1.txt`.

## Step 9: See the Conflict

GitHub should show that the branch has conflicts.

This is expected.

We created this on purpose by editing the same line in two places:

```
main branch: Chapter 1: A Different Beginning  
revise-chapter1-opening branch: Chapter 1: The Adventure Begins
```

## Step 10: Resolve the Conflict on GitHub

Click **Resolve conflicts**.

You may see something like:

```
<<<<<< main
Chapter 1: A Different Beginning
=====
Chapter 1: The Adventure Begins
>>>>>> revise-chapter1-opening

Once upon a time, a new Git repo was born.
```

Your job is to edit this into the final version you want.

## Step 11: Choose the Final Text

For example, choose:

```
Chapter 1: The Adventure Begins  
Once upon a time, a new Git repo was born.
```

Delete the conflict markers:

```
<<<<<< main  
=====  
>>>>>> revise-chapter1-opening
```

Then click **Mark as resolved** and **Commit merge**.



## Step 12: Merge the Pull Request

Once the conflict is resolved, GitHub should allow the PR to merge.

Click:

```
Merge pull request  
Confirm merge
```

Now `main` on GitHub contains the final resolved version of `chapter1.txt`.

## Step 13: Pull the Result Back Locally

Return to your terminal.

Switch to `main`:

```
git switch main
```

Pull the merged result from GitHub:

```
git pull
```

Open `chapter1.txt` and confirm that the final opening line is there.

## Step 14: Check the History

Use:

```
git log --oneline --graph --decorate --all
```

You should see a history that includes:

- the original `chapter1.txt` commit
- your branch commit
- the direct edit on `main`
- the conflict resolution / merge from GitHub

## Optional Cleanup

Delete the local branch after the Pull Request is merged:

```
git branch -d revise-chapter1-opening
```

If GitHub offers to delete the remote branch after merging, you can click:

Delete branch

# GitHub as Portfolio

# GitHub Portfolio Tips

A strong GitHub profile should make your work easy to understand.

Good habits:

- Pin your best repositories
- Write clear README files
- Use meaningful repository names
- Keep folders organized
- Commit regularly with readable messages
- Do not upload passwords, API keys, or private data
- Make sure your project can be understood by your target audience

# A Good README Is Your Project's Front Door

When someone opens your repository, the README is usually the first thing they see.

A good README answers:

- What is this project?
- Why does it matter?
- What files are included?
- How do I run it?
- What tools or packages are needed?
- What did you learn or produce?

# Recap



## What We Practiced

```
git branch           # list branches
git switch -c new-branch # create and switch
git status           # check current state
git diff             # inspect changes
git add file         # stage changes
git commit -m "message" # save local checkpoint
git push -u origin branch # send branch to GitHub
git pull             # bring GitHub changes local
```

And on GitHub:

```
Open Pull Request
Resolve conflict
Merge Pull Request
```

# Big Picture

Branch = safe place to work  
Pull Request = review space on GitHub  
Merge = bring work back into main  
Conflict = GitHub needs human judgment

A conflict is not a failure.

It is a normal part of collaboration.

## Discussion

- When would you create a branch?
- Why might we prefer merging on GitHub instead of locally in this course?
- What kinds of files are likely to cause conflicts?
- How can small commits and focused branches make conflicts easier?